

Fundamentals of API Development

Reference Only: Automated API Test Suite

Superseded by the 60-minute card applications capstone

Lab Overview

Goal: Turn Lab 1.1's manual API observations into repeatable JUnit5 assertions, extending the quickstart API's test suite from one happy-path test per controller to a small suite covering negative paths, method-not-allowed handling, and content-type contracts.

- **Format:** Individual or pair work
- **Tools:** IntelliJ, Maven, the quickstart API project in `api-dev-setup`

NOTE

The DevOps overview just covered Surefire vs Failsafe and the layered test strategy. This lab occupies layer one, the cheapest, fastest gate, the same place a real pipeline runs first.

What's in the Companion Repo (1/2)

Open `day4/lab2_3_simulating_an_automated_api_test_suite/` in the companion repo.

Path	What it is
<code>start/HealthControllerTest.additions-template.java</code>	The existing test plus a worked-example negative-path test, plus two TODOs
<code>start/GreetingControllerTest.additions-template.java</code>	The existing test plus two TODOs
<code>start/NEWMAN-STRETCH.md</code>	The exact command for the Newman stretch goal
<code>solution/</code>	Completed reference versions of both real test files

What's in the Companion Repo (2/2)

SETUP

No running API needed for MockMvc tests. Copy both template files' new test methods onto the real files in `api-dev-setup/quickstart-project/src/test/java/com/apidev/quickstart/controller/`.

Step 1: Read the Worked Example

The 404 Test Is Done

Open `start/HealthControllerTest.additions-template.java`. Read `unmappedPathReturns404`, it is the worked example, the same pattern `checkAuthentication` used in Lab 2.2: already implemented, already passing, and the shape the TODOs should follow.

It turns Lab 1.1's manual observation ("an unmapped path comes back 404, not a blank 200") into an assertion that runs every time.

Step 2: Implement the Four TODOs

HealthControllerTest TODOs

Copy the new test methods from both template files onto the real files. Do not create second copies, there must be exactly one of each class.

TODO	Test	What it asserts
1	<code>postToHealthReturns405WithAllowHeader</code>	<code>POST /api/v1/health</code> returns <code>405</code> , and the <code>Allow</code> header contains <code>GET</code>
2	<code>healthResponseContentTypeIsJson</code>	<code>GET /api/v1/health</code> returns <code>Content-Type: application/json</code>

GreetingControllerTest TODOs

TODO	Test	What it asserts
3	<code>greetingHandlesNameWithSpaces</code>	GET <code>/api/v1/greetings/{name}</code> with a name containing a space still succeeds and reflects the name unmodified
4	<code>greetingWithoutNameReturns404</code>	GET <code>/api/v1/greetings/</code> with no name segment returns <code>404</code> , not a blank <code>200</code>

THESE ARE NOT GUESSES

Every one of these is a specific fact Lab 1.1's worksheet already recorded about this API. You are making an existing observation repeatable.

Step 3: Run It

Seven Tests, All Green

Run both classes together from the quickstart project root:

```
cd ../api-dev-setup/quickstart-project  
mvn test
```

You should see 4 tests pass in `HealthControllerTest` and 3 in `GreetingControllerTest`, seven total, all green.

IF ANY FAIL

Re-read the TODO comment for that test. Each one names the exact status code and header or JSON path to assert on.

Stretch Goals

If you finish early:

1. **Add more negative test cases** beyond the four required TODOs. Lab 1.1's worksheet has more raw material: try `HEAD /api/v1/health` (same status and content type as `GET`, empty body), or a greeting name that is a number instead of a word.
2. **Run the Postman collection headlessly** with Newman, simulating the "deploy & test" stage of a CI pipeline. See `start/NEWMAN-STRETCH.md` for the exact command.

Lab Completion Checklist

Checklist

- [] Worked example (`unmappedPathReturns404`) read and understood
- [] All four TODOs implemented: two in `HealthControllerTest` , two in `GreetingControllerTest`
- [] `mvn test` passes all seven tests with zero failures
- [] Both test files exist exactly once, no duplicates

DONE?

Compare your implementation against `solution/` one TODO at a time. This retired lab remains available as optional JUnit5 practice; the card applications capstone now provides the assessed test suite.

Reference Lab Complete

Continue with the card applications capstone